SCES's

INDIRA COLLEGE OF ENGINEERING &

MANAGEMENT, PUNE

Department of Computer Engineering

LABORATORY MANUAL

AY :- 2024-2025

Course Name: LAB PRACTICE V

BE – COMPUTER ENGINEERING

SEMESTER-II

Course Code: 410255

TEACHING SCHEME	EXAMINATION SCHEME
Practical: 2 Hrs/Week	Term Work: 50 Marks Practical : 50 Marks

Name of Faculty

Prof. Shraddha Suryawanshi

INDEX

Sr. No.	Title	Page Number
	Design and implement Parallel Breadth First Search and Depth First Search based on existing algorithms using OpenMP. Use a Tree or an undirected graph for BFS and DFS	
1	Write a program to implement Parallel Bubble Sort and Merge sort using OpenMP. Use existing algorithms and measure the performance of sequential and parallel algorithms.	
2	Implement Min, Max, Sum and Average operations using Parallel Reduction.	
3	Write a CUDA Program for : 1. Addition of two large vectors 2. Matrix Multiplication using CUDA C	
4	**1.** Mini Project: Evaluate performance enhancement of parallel Quicksort Algorithm using MPI **OR** **2.** Mini Project: Implement Huffman Encoding on GPU **OR** **3**. Mini Project: Implement Parallelization of Database Query optimization **OR** **4.** Mini Project: Implement Non-Serial Polyadic Dynamic Programming with GPU Parallelization	
5	**Linear regression by using Deep Neural network:** Implement Boston housing price predictionproblem by Linear regression using Deep Neural network. Use Boston House price predictiondataset.	
	Classification using Deep neural network (Any One from the following) 1. Multiclass classification using Deep Neural Networks: Example: Use the OCR letter recognition datasethttps://archive.ics.uci.edu/ml/datasets/letter+recognition 2. Binary classification using Deep Neural Networks Example: Classify movie reviews into positive" reviews and "negative" reviews, just based on the text content of the reviews. Use IMDB dataset	
1	Convolutional neural network (CNN) (Any One from the following) Use any dataset of plant disease and design a plant disease detection system using CNN. □ Use MNIST Fashion Dataset and create a classifier to classify fashion clothing into categories.	
2	**1**. Mini Project: Human Face Recognition **OR** **2**. Mini Project: Gender and Age Detection: predict if a person is a male or female and also their age **OR** **3**. Mini Project: Colorizing Old B&W Images: color old black and white images to colorful images	

3	Design and implement Parallel Breadth First Search and Depth First Search based on existing algorithms using OpenMP. Use a Tree or an undirected graph for BFS and DFS	
4	Write a program to implement Parallel Bubble Sort and Merge sort using OpenMP. Use existing algorithms and measure the performance of sequential and parallel algorithms.	
5	Implement Min, Max, Sum and Average operations using Parallel Reduction.	

Course Objectives:

- To understand and implement searching and sorting algorithms.
- To learn the fundamentals of GPU Computing in the CUDA environment.
- To illustrate the concepts of Artificial Intelligence/Machine Learning(AI/ML).
- To understand Hardware acceleration.
- To implement different deep learning models.

Course Outcomes:

CO1: Analyze and measure performance of sequential and parallel algorithms.
CO2: Design and Implement solutions for multicore/Distributed/parallel environment.
CO3: Identify and apply the suitable algorithms to solve AI/ML problems.
CO4: Apply the technique of Deep Neural network for implementing Linear regression andclassification.
CO5: Apply the technique of Convolution (CNN) for implementing Deep Learning models.
CO6: Design and develop Recurrent Neural Network (RNN) for prediction.

Guidelines for Laboratory Conduction

● List of recommended programming assignments and sample mini-projects is provided for
reference.
● Referring these, Course Teacher or Lab Instructor may frame the assignments/mini-project by
understanding the prerequisites, technological aspects, utility and recent trends related tothe
respective courses.
● Preferably there should be multiple sets of assignments/mini-project and distribute among
batches of students.
● Real world problems/application based assignments/mini-projects create interest among
learners serving as foundation for future research or startup of business projects.
● Mini-project can be completed in group of 2 to 3 students.
Software Engineering approach with proper documentation is to be strictly followed.
● Use of open source software is to be encouraged.
● Instructor may also set one assignment or mini-project that is suitable to respective course
beyond the scope of syllabus.
Operating System recommended :- 64-bit Open source Linux or its derivative
Programming Languages: Object Oriented Languages
C++/JAVA/PYTHON/R
Programming tools recommended: Front End: Java/Perl/PHP/Python/Ruby/.net, Backend :

MongoDB/MYSQL/Oracle, Database Connectivity : ODBC/JDBC

ASSIGNMENT NO: 1

TITLE: Design and implement Parallel Breadth First Search and Depth First Search based on existing algorithms using OpenMP. Use a Tree or an undirected graph for BFS and DFS

OBJECTIVE: Students should be able to perform Parallel Breadth First Search & Deapth First Search based on existing algorithms using OpenMP

THEORY:

What is BFS?

BFS stands for Breadth-First Search. It is a graph traversal algorithm used to explore all the nodes of a graph or tree systematically, starting from the root node or a specified starting point, and visiting all the neighboring nodes at the current depth level before moving on to the next depth level. The algorithm uses a queue data structure to keep track of the nodes that need to be visited, and marks each visited node to avoid processing it again. The basic idea of the BFS algorithm is to visit all the nodes at a given level before moving on to the next level, which ensures that all the nodes are visited in breadth-first order. BFS is commonly used in many applications, such as finding the shortest path between two nodes, solving puzzles, and searching through a tree or graph.

Example of BFS

Now let's take a look at the steps involved in traversing a graph by using Breadth-First Search:

Step 1: Take an Empty Queue.

Step 2: Select a starting node (visiting a node) and insert it into the Queue.

Step 3: Provided that the Queue is not empty, extract the node from the Queue and insert its child nodes (exploring a node) into the Queue.

Step 4: Print the extracted node.

How Parallel BFS Work

● Parallel BFS (Breadth-First Search) is an algorithm used to explore all the nodes of a graph or tree systematically in parallel. It is a popular parallel algorithm used for graph traversal in distributed computing, shared-memory systems, and parallel clusters.

● The parallel BFS algorithm starts by selecting a root node or a specified starting point, and then assigning it to a thread or processor in the system. Each thread maintains a local queue of nodes to be visited and marks each visited node to avoid processing it again.

● The algorithm then proceeds in levels, where each level represents a set of nodes that are at a certain distance from the root node. Each thread processes the nodes in its local

queue at the current level, and then exchanges the nodes that are adjacent to the current level with other threads or processors.This is done to ensure that the nodes at the next level are visited by the next iteration of the algorithm.

● The parallel BFS algorithm uses two phases: the computation phase and the communication phase. In the computation phase, each thread processes the nodes in its local queue, while in the communication phase, the threads exchange the nodes that are adjacent to the current level with other threads or processors.

● The parallel BFS algorithm terminates when all nodes have been visited or when a specified node has been found. The result of the algorithm is the set of visited nodes or the shortest path from the root node to the target node.

● Parallel BFS can be implemented using different parallel programming models, such as OpenMP, MPI, CUDA, and others. The performance of the algorithm depends on the number of threads or processors used, the size of the graph, and the communication overhead between the threads or processors.

What is DFS?

DFS stands for Depth-First Search. It is a popular graph traversal algorithm that explores as far as possible along each branch before backtracking. This algorithm can be used to find the shortest path between two vertices or to traverse a graph in a systematic way. The algorithm starts at the root node
and explores as far as possible along each branch before backtracking. The backtracking is done to
explore the next branch that has not been explored yet.

DFS can be implemented using either a recursive or an iterative approach. The recursive approach is simpler to implement but can lead to a stack overflow error for very large graphs. The iterative approach uses a stack to keep track of nodes to be explored and is preferred for larger graphs.

DFS can also be used to detect cycles in a graph. If a cycle exists in a graph, the DFS algorithm will eventually reach a node that has already been visited, indicating that a cycle exists. A standard DFS implementation puts each vertex of the graph into one of two categories:

1. Visited
2. Not Visited

The purpose of the algorithm is to mark each vertex as visited while avoiding cycles.

Example of DFS:

To implement DFS traversal, you need to take the following stages.

Step 1: Create a stack with the total number of vertices in the graph as the size.

Step 2: Choose any vertex as the traversal's beginning point. Push a visit to that vertex and add it to the stack.

Step 3 - Push any non-visited adjacent vertices of a vertex at the top of the stack to the top of the stack.

Step 4 - Repeat steps 3 and 4 until there are no more vertices to visit from the vertex at the top of the stack.

Step 5 - If there are no new vertices to visit, go back and pop one from the stack using backtracking.
Step 6 - Continue using steps 3, 4, and 5 until the stack is empty.
Step 7 - When the stack is entirely unoccupied, create the final spanning tree by deleting the graph's unused edges.
Consider the following graph as an example of how to use the dfs algorithm.

How Parallel DFS Work

● Parallel Depth-First Search (DFS) is an algorithm that explores the depth of a graph structure to search for nodes. In contrast to a serial DFS algorithm that explores nodes in a sequential manner, parallel DFS algorithms explore nodes in a parallel manner, providing a significant speedup in large graphs.
● Parallel DFS works by dividing the graph into smaller subgraphs that are explored simultaneously. Each processor or thread is assigned a subgraph to explore, and they work independently to explore the subgraph using the standard DFS algorithm. During the exploration process, the nodes are marked as visited to avoid revisiting them.
● To explore the subgraph, the processors maintain a stack data structure that stores the nodes in the order of exploration. The top node is picked and explored, and its adjacent nodes are pushed onto the stack for further exploration. The stack is updated concurrently by the processors as they explore their subgraphs.
● Parallel DFS can be implemented using several parallel programming models such as OpenMP, MPI, and CUDA. In OpenMP, the #pragma omp parallel for directive is used to distribute the work among multiple threads. By using this directive, each thread operates on a different part of the graph, which increases the performance of the DFS algorithm.

Concept of OpenMP
● OpenMP (Open Multi-Processing) is an application programming interface (API) that supports shared-memory parallel programming in C, C++, and Fortran. It is used to write parallel programs that can run on multicore processors, multiprocessor systems, and parallel computing clusters.
● OpenMP provides a set of directives and functions that can be inserted into the source code of a program to parallelize its execution. These directives are simple and easy to use, and they can be applied to loops, sections, functions, and other program constructs. The compiler then generates parallel code that can run on multiple processors concurrently.
● OpenMP programs are designed to take advantage of the shared-memory architecture of modern processors, where multiple processor cores can access the same memory. OpenMP uses a fork-join model of parallel execution, where a master thread forks multiple worker threads to execute a parallel region of the code, and then waits for all threads to complete before continuing with the sequential part of the code.
● OpenMP is widely used in scientific computing, engineering, and other fields that require high-performance computing. It is supported by most modern compilers and is

available on a wide range of platforms, including desktops, servers, and supercomputers.

CODE:

BFS:

```
#include<iostream>
#include<stdlib.h>
#include<queue>
using namespace std;


class node
{
   public:

    node *left, *right;
    int data;

};

class Breadthfs
{

 public:

 node *insert(node *, int);
 void bfs(node *);

};


node *insert(node *root, int data)
// inserts a node in tree
{

    if(!root)
    {

    root=new node;
    root->left=NULL;
    root->right=NULL;
```

```
root->data=data;
return root;
}

queue<node *> q;
q.push(root);

while(!q.empty())
{

node *temp=q.front();
q.pop();

if(temp->left==NULL)
{

temp->left=new node;
temp->left->left=NULL;
temp->left->right=NULL;
temp->left->data=data;
return root;
}
else
{

q.push(temp->left);

}

if(temp->right==NULL)
{

temp->right=new node;
temp->right->left=NULL;
temp->right->right=NULL;
temp->right->data=data;
return root;
}
else
{

q.push(temp->right);

}

}
```

```
}


void bfs(node *head)
{
#include <cuda_runtime.h>
#include <iostream>
__global__ void matmul(int* A, int* B, int* C, int N) {
int Row = blockIdx.y*blockDim.y+threadIdx.y;
int Col = blockIdx.x*blockDim.x+threadIdx.x;
if (Row < N && Col < N) {
int Pvalue = 0;
for (int k = 0; k < N; k++) {
Pvalue += A[Row*N+k] * B[k*N+Col];
}
C[Row*N+Col] = Pvalue;
}
}
int main() {
int N = 512;
int size = N * N * sizeof(int);
int* A, * B, * C;
int* dev_A, * dev_B, * dev_C;
cudaMallocHost(&A, size);
cudaMallocHost(&B, size);
cudaMallocHost(&C, size);
cudaMalloc(&dev_A, size);
cudaMalloc(&dev_B, size);
cudaMalloc(&dev_C, size);
// Initialize matrices A and B
for (int i = 0; i < N; i++) {
for (int j = 0; j < N; j++) {
A[i*N+j] = i*N+j;
B[i*N+j] = j*N+i;
}
}
cudaMemcpy(dev_A, A, size,
cudaMemcpyHostToDevice);
cudaMemcpy(dev_B, B, size,
cudaMemcpyHostToDevice);
dim3 dimBlock(16, 16);
dim3 dimGrid(N/dimBlock.x, N/dimBlock.y);
matmul<<<dimGrid, dimBlock>>>(dev_A, dev_B,
dev_C, N);
cudaMemcpy(C, dev_C
// Print the result
for (int i = 0; i < 10; i++) {
for (int j = 0; j < 10; j++) {
std::cout << C[i*N+j] << " ";
}
std::cout << std::endl;
}
// Free memory
```

```
cudaFree(dev_A);
cudaFree(dev_B);
cudaFree(dev_C);
cudaFreeHost(A);
cudaFreeHost(B);
cudaFreeHost(C);
return 0;

    queue<node*> q;
    q.push(head);

    int qSize;

    while (!q.empty())
    {
    qSize = q.size();
    #pragma omp parallel for
            //creates parallel threads
    for (int i = 0; i < qSize; i++)
    {
    node* currNode;
    #pragma omp critical
    {
      currNode = q.front();
      q.pop();
      cout<<"\t"<<currNode->data;

    }// prints parent node
    #pragma omp critical
    {
    if(currNode->left)// push parent's left node in queue
    q.push(currNode->left);
    if(currNode->right)
    q.push(currNode->right);
    }// push parent's right node in queue

    }
    }

}

int main(){

    node *root=NULL;
    int data;
    char ans;
```

```
    do
    {
    cout<<"\n enter data=>";
    cin>>data;

    root=insert(root,data);

    cout<<"do you want insert one more node?";
    cin>>ans;

    }while(ans=='y'||ans=='Y');

    bfs(root);

    return 0;
}
```

Run Commands:

1.

```
g++ -fopenmp bfs.cpp -o bfs
```

2.

1.

```
./bfs
```

2.

Output:
This code represents a breadth-first search (BFS) algorithm on a binary tree using OpenMP for parallelization. The program asks for user input to insert nodes into the binary tree and then performs the BFS algorithm using multiple threads. Here's an example output for a binary tree with nodes 5, 3, 2, 1, 7, and 8:

DFS:

```
#include <iostream>
#include <vector>
#include <stack>
#include <omp.h>

using namespace std;

const int MAX = 100000;
vector<int> graph[MAX];
bool visited[MAX];

void dfs(int node) {
stack<int> s;
s.push(node);

while (!s.empty()) {
    int curr_node = s.top();
    s.pop();

    if (!visited[curr_node]) {
        visited[curr_node] = true;

        if (visited[curr_node]) {
        cout << curr_node << " ";
    }

        #pragma omp parallel for
        for (int i = 0; i < graph[curr_node].size(); i++) {
            int adj_node = graph[curr_node][i];
            if (!visited[adj_node]) {
                s.push(adj_node);
            }
        }
    }
}
}

int main() {
int n, m, start_node;
cout << "Enter No of Node,Edges,and start node:" ;
cin >> n >> m >> start_node;
        //n: node,m:edges

cout << "Enter Pair of edges:" ;
```

```
for (int i = 0; i < m; i++) {
    int u, v;

    cin >> u >> v;
//u and v: Pair of edges
    graph[u].push_back(v);
    graph[v].push_back(u);
}

#pragma omp parallel for
for (int i = 0; i < n; i++) {
    visited[i] = false;
}

dfs(start_node);

/* for (int i = 0; i < n; i++) {
    if (visited[i]) {
        cout << i << " ";
    }
}*/

return 0;
}
```

Conclusion-

In this way we can achieve parallelism while implementing BFS & DFS.

ASSIGNMENT NO: 2

TITLE: Write a program to implement Parallel Bubble Sort and Merge sort using OpenMP. Use existing algorithms and measure the performance of sequential and parallel algorithms.

OBJECTIVE: Students should be able to Write a program to implement Parallel Bubble Sort and can measure the performance of sequential and parallel algorithms..

THEORY:

What is Bubble Sort?

Bubble Sort is a simple sorting algorithm that works by repeatedly swapping adjacent elements if they are in the wrong order. It is called "bubble" sort because the algorithm moves the larger elements towards the end of the array in a manner that resembles the rising of bubbles in a liquid.
The basic algorithm of Bubble Sort is as follows:
1. Start at the beginning of the array.
2. Compare the first two elements. If the first element is greater than the second element, swap them.
3. Move to the next pair of elements and repeat step 2.
4. Continue the process until the end of the array is reached.
5. If any swaps were made in step 2-4, repeat the process from step 1.
The time complexity of Bubble Sort is O(n^2), which makes it inefficient for large lists. However, it has the advantage of being easy to understand and implement, and it is useful for educational purposes and for sorting small datasets. Bubble Sort has limited practical use in modern software development due to its inefficient time complexity of O(n^2) which makes it unsuitable for sorting large datasets. However, Bubble Sort has some advantages and use cases that make it a valuable algorithm to understand, such as:
1. Simplicity: Bubble Sort is one of the simplest sorting algorithms, and it is easy to understand and implement. It can be used to introduce the concept of sorting to beginners and as a basis for more complex sorting algorithms.
2. Educational purposes: Bubble Sort is often used in academic settings to teach the principles of sorting algorithms and to help students understand how algorithms work.
3. Small datasets: For very small datasets, Bubble Sort can be an efficient sorting algorithm, as its overhead is relatively low.
4. Partially sorted datasets: If a dataset is already partially sorted, Bubble Sort can be very efficient. Since Bubble Sort only swaps adjacent elements that are in the wrong order, it has a low number of operations for a partially sorted dataset.
5. Performance optimization: Although Bubble Sort itself is not suitable for sorting large datasets, some of its techniques can be used in combination with other sorting algorithms to optimize their performance. For example, Bubble Sort can be used to optimize the performance of Insertion Sort by reducing the number of comparisons needed.

Example of Bubble sort

Let's say we want to sort a series of numbers 5, 3, 4, 1, and 2 so that they are arranged in ascending order… The sorting begins the first iteration by comparing the first two values. If the first value is greater than the second, the algorithm pushes the first value to the index of the second value.

How Parallel Bubble Sort Work:

● Parallel Bubble Sort is a modification of the classic Bubble Sort algorithm that takes advantage of parallel processing to speed up the sorting process.

● In parallel Bubble Sort, the list of elements is divided into multiple sublists that are sorted concurrently by multiple threads. Each thread sorts its sublist using the regular Bubble Sort algorithm. When all sublists have been sorted, they are merged together to form the final sorted
list.

● The parallelization of the algorithm is achieved using OpenMP, a programming API that supports parallel processing in C++, Fortran, and other programming languages. OpenMP provides a set of compiler directives that allow developers to specify which parts of the code can be executed in parallel.

● In the parallel Bubble Sort algorithm, the main loop that iterates over the list of elements is divided into multiple iterations that are executed concurrently by multiple threads. Each thread sorts a subset of the list, and the threads synchronize their work at the end of each iteration to ensure that the elements are properly ordered.

● Parallel Bubble Sort can provide a significant speedup over the regular Bubble Sort algorithm, especially when sorting large datasets on multi-core processors. However, by the overhead of thread creation and synchronization, and it may not be worth the effort for small datasets or when using a single-core processor.

How to measure the performance of sequential and parallel algorithms?

To measure the performance of sequential Bubble sort and parallel Bubble sort algorithms, you can,follow these steps:

1. Implement both the sequential and parallel Bubble sort algorithms.
2. Choose a range of test cases, such as arrays of different sizes and different degrees of sortedness,to test the performance of both algorithms.
3.Use a reliable timer to measure the execution time of each algorithm on each test case.
4. Record the execution times and analyze the results.

When measuring the performance of the parallel Bubble sort algorithm, you will need to specify the number of threads to use. You can experiment with different numbers of threads to find the optimal value for your system.

What is Merge Sort?

Merge sort is a sorting algorithm that uses a divide-and-conquer approach to sort an array or a list of elements. The algorithm works by recursively dividing the input array into two halves, sorting each half, and then merging the sorted halves to produce a sorted

output.
The merge sort algorithm can be broken down into the following steps:
1. Divide the input array into two halves.
2. Recursively sort the left half of the array.
3. Recursively sort the right half of the array.
4. Merge the two sorted halves into a single sorted output array.

● The merging step is where the bulk of the work happens in merge sort. The algorithm compares the first elements of each sorted half, selects the smaller element, and appends it to the output array. This process continues until all elements from both halves have been appended to the output array.

● The time complexity of merge sort is O(n log n), which makes it an efficient sorting algorithm for large input arrays. However, merge sort also requires additional memory to store the output array, which can make it less suitable for use with limited memory resources.

● In simple terms, we can say that the process of merge sort is to divide the array into two halves, sort each half, and then merge the sorted halves back together. This process is repeated until the entire array is sorted.

● One thing that you might wonder is what is the specialty of this algorithm. We already have a number of sorting algorithms then why do we need this algorithm? One of the main advantages of merge sort is that it has a time complexity of O(n log n), which means it can sort large arrays relatively quickly. It is also a stable sort, which means that the order of elements with equal values is preserved during the sort.

● Merge sort is a popular choice for sorting large datasets because it is relatively efficient and easy to implement. It is often used in conjunction with other algorithms, such as quicksort, to improve the overall performance of a sorting routine.

● Parallel merge sort is a parallelized version of the merge sort algorithm that takes advantage of multiple processors or cores to improve its performance. In parallel merge sort, the input array is divided into smaller subarrays, which are sorted in parallel using multiple processors or cores. The sorted subarrays are then merged together in parallel to produce the final sorted output.

● The parallel merge sort algorithm can be broken down into the following steps:

Divide the input array into smaller subarrays.

● Assign each subarray to a separate processor or core for sorting.

● Sort each subarray in parallel using the merge sort algorithm.

● Merge the sorted subarrays together in parallel to produce the final sorted output.

● The merging step in parallel merge sort is performed in a similar way to the merging step in the sequential merge sort algorithm. However, because the subarrays are sorted in parallel, the merging step can also be performed in parallel using multiple processors or cores. This can significantly reduce the time required to merge the sorted subarrays and produce the final output.

● Parallel merge sort can provide significant performance benefits for large input arrays with many elements, especially when running on hardware with multiple processors or cores. However, it also requires additional overhead to manage the parallelization, and may not always provide performance improvements for smaller input sizes or when run on hardware with limited parallel processing capabilities.

CODE:

Title: Write a program to implement Parallel Bubble Sort using OpenMP

```
#include<iostream>
#include<stdlib.h>
#include<omp.h>
using namespace std;

void bubble(int *, int);
void swap(int &, int &);


void bubble(int *a, int n)
{
   for(  int i = 0;  i < n;  i++ )
    {
   int first = i % 2;

   #pragma omp parallel for shared(a,first)
   for(  int j = first;  j < n-1;  j += 2  )
     {
   if(  a[ j ]  >  a[ j+1 ]  )
     {
     swap(  a[ j ],  a[ j+1 ]  );
     }
     }
    }
}


void swap(int &a, int &b)
{

   int test;
   test=a;
   a=b;
   b=test;

}

int main()
{

   int *a,n;
```

```
    cout<<"\n enter total no of elements=>";
    cin>>n;
    a=new int[n];
    cout<<"\n enter elements=>";
    for(int i=0;i<n;i++)
    {
    cin>>a[i];
    }

    bubble(a,n);

    cout<<"\n sorted array is=>";
    for(int i=0;i<n;i++)
    {
    cout<<a[i]<<endl;
    }


return 0;
}
```

MERGE SORT

```
#include<iostream>
#include<stdlib.h>
#include<omp.h>
using namespace std;


void mergesort(int a[],int i,int j);
void merge(int a[],int i1,int j1,int i2,int j2);

void mergesort(int a[],int i,int j)
{
int mid;
if(i<j)
{
    mid=(i+j)/2;
```

```
    #pragma omp parallel sections
    {

        #pragma omp section
        {
            mergesort(a,i,mid);
        }

        #pragma omp section
        {
            mergesort(a,mid+1,j);
        }
    }

    merge(a,i,mid,mid+1,j);
}

}

void merge(int a[],int i1,int j1,int i2,int j2)
{
int temp[1000];
int i,j,k;
i=i1;
j=i2;
k=0;

while(i<=j1 && j<=j2)
{
    if(a[i]<a[j])
    {
        temp[k++]=a[i++];
    }
    else
    {
        temp[k++]=a[j++];
   }
}

while(i<=j1)
{
    temp[k++]=a[i++];
}

while(j<=j2)
```

```
{
    temp[k++]=a[j++];
}

for(i=i1,j=0;i<=j2;i++,j++)
{
    a[i]=temp[j];
}
}


int main()
{
int *a,n,i;
cout<<"\n enter total no of elements=>";
cin>>n;
a= new int[n];

cout<<"\n enter elements=>";
for(i=0;i<n;i++)
{
    cin>>a[i];
}
  // start=.......
//#pragma omp…..
mergesort(a, 0, n-1);
//          stop…….
cout<<"\n sorted array is=>";
for(i=0;i<n;i++)
{
    cout<<"\n"<<a[i];
}
  // Cout<<Stop-Start
return 0;
}
```

Second Code:

```
#include <iostream>
#include <omp.h>

void merge(int* arr, int l, int m, int r) {
int i, j, k;
int n1 = m - l + 1;
int n2 = r - m;
```

```
int L[n1], R[n2];

for (i = 0; i < n1; i++)
    L[i] = arr[l + i];
for (j = 0; j < n2; j++)
    R[j] = arr[m + 1 + j];

i = 0;
j = 0;
k = l;
while (i < n1 && j < n2) {
    if (L[i] <= R[j]) {
        arr[k] = L[i];
        i++;
    }
    else {
        arr[k] = R[j];
        j++;
    }
    k++;
}

while (i < n1) {
    arr[k] = L[i];
    i++;
    k++;
}

while (j < n2) {
    arr[k] = R[j];
    j++;
    k++;
}
}

void mergeSort(int* arr, int l, int r) {
if (l < r) {
    int m = l + (r - l) / 2;
    #pragma omp parallel sections
    {
        #pragma omp section
        {
            mergeSort(arr, l, m);
        }
        #pragma omp section
        {
```

```
            mergeSort(arr, m + 1, r);
        }
    }

    merge(arr, l, m, r);
}
}

int main() {
int arr[] = { 12, 11, 13, 5, 6, 7 };
int n = sizeof(arr) / sizeof(arr[0]);
  double start, stop;
std::cout << "Given array is: ";
for (int i = 0; i < n; i++)
    std::cout << arr[i] << " ";
std::cout << std::endl;
 start = omp_get_wtime();
  #pragma omp parallel
  {
mergeSort(arr, 0, n - 1);
}
  stop = omp_get_wtime();
std::cout << "Sorted array is: ";
for (int i = 0; i < n; i++)
    std::cout << arr[i] << " ";
std::cout << std::endl;

std::cout<<stop-start;
return 0;
}
```

Conclusion-

In this way we can implement Merge Sort in parallel way using OpenMP also come to know how to how to measure performance of serial and parallel algorithm

ASSIGNMENT NO: 3

TITLE: Implement Min, Max, Sum and Average operations using Parallel Reduction Use Case for various scenarios.

OBJECTIVE: To understand the concept of parallel reduction and how it can be used to perform basic mathematical operations on given data sets.

THEORY:

Parallel Reduction.

Here's a function-wise manual on how to understand and run the sample C++ program that demonstrates how to implement Min, Max, Sum, and Average operations using parallel reduction.

1. Min_Reduction function

 The function takes in a vector of integers as input and finds the minimum value in the vector using parallel reduction.The OpenMP reduction clause is used with the "min" operator to find the minimum value across all threads. The minimum value found by each thread is reduced to the overall minimum value of the
entire array. The final minimum value is printed to the console.

2. Max_Reduction function

 The function takes in a vector of integers as input and finds the maximum value in the vector using parallel reduction. The OpenMP reduction clause is used with the "max" operator to find the maximum value
across all threads. The maximum value found by each thread is reduced to the overall maximum value of the entire array. The final maximum value is printed to the console.

3. Sum_Reduction function

 The function takes in a vector of integers as input and finds the sum of all the values in the vector using parallel reduction. The OpenMP reduction clause is used with the "+" operator to find the sum across all threads. The sum found by each thread is reduced to the overall sum of the entire array.The final sum is printed to the console.

4. Average_Reduction function

 The function takes in a vector of integers as input and finds the average of all the values in the vector using parallel reduction.The OpenMP reduction clause is used with the "+" operator to find the sum across all threads. The sum found by each thread is reduced to the overall sum of the entire array. The final sum is divided by the size of the array to find the average. The final average value is printed to the console.

5. Main Function

 The function initializes a vector of integers with some values.The function calls the min_reduction, max_reduction, sum_reduction, and average_reduction functions on the input vector to find the corresponding values. The final minimum, maximum, sum, and average values are printed to the console.

6. Compiling and running the program

Compile the program: You need to use a C++ compiler that supports OpenMP, such as g++ or clang. Open a terminal and navigate to the directory where your program is saved. Then, compilethe program using the following command:
$ g++ -fopenmp program.cpp -o program
This command compiles your program and creates an executable file named "program". The "-fopenmp" flag tells the compiler to enable OpenMP.
Run the program: To run the program, simply type the name of the executable file in the terminal and press Enter:

$ ./program

Implement Min, Max, Sum and Average operations using Parallel Reduction.

```
#include <iostream>
//#include <vector>
#include <omp.h>
#include <climits>
using namespace std;
void min_reduction(int arr[], int n) {
  int min_value = INT_MAX;
  #pragma omp parallel for reduction(min: min_value)
  for (int i = 0; i < n; i++) {
if (arr[i] < min_value) {
   min_value = arr[i];
}
  }
  cout << "Minimum value: " << min_value << endl;
}

void max_reduction(int arr[], int n) {
  int max_value = INT_MIN;
  #pragma omp parallel for reduction(max: max_value)
  for (int i = 0; i < n; i++) {
if (arr[i] > max_value) {
   max_value = arr[i];
}
  }
  cout << "Maximum value: " << max_value << endl;
}

void sum_reduction(int arr[], int n) {
  int sum = 0;
```

```
   #pragma omp parallel for reduction(+: sum)
   for (int i = 0; i < n; i++) {
sum += arr[i];
  }
  cout << "Sum: " << sum << endl;
}

void average_reduction(int arr[], int n) {
  int sum = 0;
  #pragma omp parallel for reduction(+: sum)
  for (int i = 0; i < n; i++) {
sum += arr[i];
  }
  cout << "Average: " << (double)sum / (n-1) << endl;
}

int main() {
    int *arr,n;
    cout<<"\n enter total no of elements=>";
    cin>>n;
    arr=new int[n];
    cout<<"\n enter elements=>";
    for(int i=0;i<n;i++)
    {
    cin>>arr[i];
    }

//   int arr[] = {5, 2, 9, 1, 7, 6, 8, 3, 4};
//   int n = size(arr);

  min_reduction(arr, n);
  max_reduction(arr, n);
  sum_reduction(arr, n);
  average_reduction(arr, n);
}
```

ASSIGNMENT NO: 4

TITLE: Write a CUDA Program for : 1. Addition of two large vectors
2. Matrix Multiplication using CUDA C

OBJECTIVE: Students should be able to perform CUDA Program for Addition of two large vectors & Matrix multipication Using CUDA C

THEORY:

What is CUDA

CUDA (Compute Unified Device Architecture) is a parallel computing platform and programming model developed by NVIDIA. It allows developers to use the power of NVIDIA graphics processing units (GPUs) ,to accelerate computation tasks in various applications, including scientific computing, machine learning, and computer vision.CUDA provides a set of programming APIs, libraries, and tools that enable developers to write and execute parallel code on NVIDIA GPUs. It supports popular programming languages like C, C++, and Python, and provides a simple programming model that abstracts away much of the low-level details of GPU architecture. Using CUDA, developers can exploit the massive parallelism and high computational power of GPUs to accelerate computationally intensive tasks, such as matrix operations, image processing, and deep learning. CUDA has become an important tool for scientific research and is widely used in fields like physics, chemistry, biology, and engineering.

Steps for Addition of two large vectors using CUDA

1. Define the size of the vectors: In this step, you need to define the size of the vectors that you want to add. This will determine the number of threads and blocks you will need to use to parallelize the
addition operation.
2. Allocate memory on the host: In this step, you need to allocate memory on the host for the two vectors that you want to add and for the result vector. You can use the C malloc function to allocate memory.
3. Initialize the vectors: In this step, you need to initialize the two vectors that you want to add on the host. You can use a loop to fill the vectors with data.
4. Allocate memory on the device: In this step, you need to allocate memory on the device for the two vectors that you want to add and for the result vector. You can use the CUDA function cudaMalloc to allocate memory.
5. Copy the input vectors from host to device: In this step, you need to copy the two input vectors from the host to the device memory. You can use the CUDA function cudaMemcpy to copy the vectors.
6. Launch the kernel: In this step, you need to launch the CUDA kernel that will perform the addition operation. The kernel will be executed by multiple threads in parallel. You can use the <<<...>>>
syntax to specify the number of blocks and threads to use.
7. Copy the result vector from device to host: In this step, you need to copy the result

vector from the device memory to the host memory. You can use the CUDA function cudaMemcpy to copy the result vector.
7. Free memory on the device: In this step, you need to free the memory that was allocated on the device. You can use the CUDA function cudaFree to free the memory.
9. Free memory on the host: In this step, you need to free the memory that was allocated on the host. You can use the C free function to free the memory. Execution of Program over CUDA Environment

Steps for Matrix Multiplication using CUDA C

Here are the steps for implementing matrix multiplication using CUDA C:
1. Matrix Initialization: The first step is to initialize the matrices that you want to multiply. You can use standard C or CUDA functions to allocate memory for the matrices and initialize their values. The matrices are usually represented as 2D arrays.
2. Memory Allocation: The next step is to allocate memory on the host and the device for the matrices. You can use the standard C malloc function to allocate memory on the host and the CUDA function cudaMalloc() to allocate memory on the device.
3. Data Transfer: The third step is to transfer data between the host and the device. You can use the CUDA function cudaMemcpy() to transfer data from the host to the device or vice versa.
4. Kernel Launch: The fourth step is to launch the CUDA kernel that will perform the matrix multiplication on the device. You can use the <<<...>>> syntax to specify the number of blocks and threads to use. Each thread in the kernel will compute one element of the output matrix.
5. Device Synchronization: The fifth step is to synchronize the device to ensure that all kernel executions have completed before proceeding. You can use the CUDA function cudaDeviceSynchronize() to synchronize the device.
6. Data Retrieval: The sixth step is to retrieve the result of the computation from the device to the host. You can use the CUDA function cudaMemcpy() to transfer data from the device to the host.
7. Memory Deallocation: The final step is to deallocate the memory that was allocated on the host and the device. You can use the C free function to deallocate memory on the host and the CUDA function cudaFree() to deallocate memory on the device.

Here are the steps to run a CUDA program for adding two large vectors:

1. Install CUDA Toolkit: First, you need to install the CUDA Toolkit on your system. You can download the CUDA Toolkit from the NVIDIA website and follow the installation instructions provided.
2. Set up CUDA environment: Once the CUDA Toolkit is installed, you need to set up the CUDA environment on your system. This involves setting the PATH and LD_LIBRARY_PATH environment variables to the appropriate directories.
3. Write the CUDA program: You need to write a CUDA program that performs the addition of two large vectors. You can use a text editor to write the program and save it

with a .cu extension.
4. Compile the CUDA program: You need to compile the CUDA program using the nvcc compiler that comes with the CUDA Toolkit. The command to compile the program is:

nvcc -o program_name program_name.cu

5.This will generate an executable program named program_name.
Run the CUDA program: Finally, you can run the CUDA program by executing the executable file generated in the previous step. The command to run the program is:

./program_name

This will execute the program and perform the addition of two large vectors.

Code:

Addition of two large vectors using CUDA

```
#include <iostream>
#include <cuda_runtime.h>
#include /usr/local/cuda/include/cuda_runtime.h

_global_void addVectors(int* A, int* B, int* C, int n)
{
int i = blockIdx.x * blockDim.x + threadIdx.x;
if (i < n)
{
    C[i] = A[i] + B[i];
}
}

int main()
{
int n = 1000000;
int* A, * B, * C;
int size = n * sizeof(int);

// Allocate memory on the host
cudaMallocHost(&A, size);
cudaMallocHost(&B, size);
cudaMallocHost(&C, size);

// Initialize the vectors
for (int i = 0; i < n; i++)
{
    A[i] = i;
    B[i] = i * 2;
}
// Allocate memory on the device
```

```
int* dev_A, * dev_B, * dev_C;
cudaMalloc(&dev_A, size);
cudaMalloc(&dev_B, size);
cudaMalloc(&dev_C, size);

// Copy data from host to device
cudaMemcpy(dev_A, A, size, cudaMemcpyHostToDevice);
cudaMemcpy(dev_B, B, size, cudaMemcpyHostToDevice);

// Launch the kernel
int blockSize = 256;
int numBlocks = (n + blockSize - 1) / blockSize;
// Copy data from device to host
cudaMemcpy(C, dev_C, size, cudaMemcpyDeviceToHost);

// Print the results
for (int i = 0; i < 10; i++)
{
    cout << C[i] << " ";
}
cout << endl;

// Free memory
cudaFree(dev_A);
cudaFree(dev_B);
cudaFree(dev_C);
cudaFreeHost(A);
cudaFreeHost(B);
cudaFreeHost(C);

return 0;
}
```

Matrix muliplication using CUDA C

```
#include <cuda_runtime.h>
#include <iostream>
__global__ void matmul(int* A, int* B, int* C, int N) {
int Row = blockIdx.y*blockDim.y+threadIdx.y;
int Col = blockIdx.x*blockDim.x+threadIdx.x;
if (Row < N && Col < N) {
int Pvalue = 0;
for (int k = 0; k < N; k++) {
Pvalue += A[Row*N+k] * B[k*N+Col];
}
C[Row*N+Col] = Pvalue;
}
}
int main() {
int N = 512;
```

```
int size = N * N * sizeof(int);
int* A, * B, * C;
int* dev_A, * dev_B, * dev_C;
cudaMallocHost(&A, size);
cudaMallocHost(&B, size);
cudaMallocHost(&C, size);
cudaMalloc(&dev_A, size);
cudaMalloc(&dev_B, size);
cudaMalloc(&dev_C, size);
// Initialize matrices A and B
for (int i = 0; i < N; i++) {
for (int j = 0; j < N; j++) {
A[i*N+j] = i*N+j;
B[i*N+j] = j*N+i;
}
}
cudaMemcpy(dev_A, A, size,
cudaMemcpyHostToDevice);
cudaMemcpy(dev_B, B, size,
cudaMemcpyHostToDevice);
dim3 dimBlock(16, 16);
dim3 dimGrid(N/dimBlock.x, N/dimBlock.y);
matmul<<<dimGrid, dimBlock>>>(dev_A, dev_B,
dev_C, N);
cudaMemcpy(C, dev_C
// Print the result
for (int i = 0; i < 10; i++) {
for (int j = 0; j < 10; j++) {
std::cout << C[i*N+j] << " ";
}
std::cout << std::endl;
}
// Free memory
cudaFree(dev_A);
cudaFree(dev_B);
cudaFree(dev_C);
cudaFreeHost(A);
cudaFreeHost(B);
cudaFreeHost(C);
return 0;
```

Group 1

Assignment No. 1

Title: Linear regression by using Deep Neural network:

Objective:

To apply the technique of Deep Neural network for implementing linear regression.

Problem Statement:

Implement Boston housing price prediction problem by linear regression using Deep Neural network. Use Boston House price prediction dataset.

Hardware Requirements:

Operating System recommended :- 64-bit Open source Linux or its derivative,

Multi-core CPU

GPU (Optional but recommended for faster training)

Sufficient RAM to accommodate the dataset and model

Software Requirements:

Programming Languages: Object Oriented Languages C++/JAVA/PYTHON/R

Tensorflow, Keras for Deep Learning

Jupyter Notebook or any Python IDE

Theory:

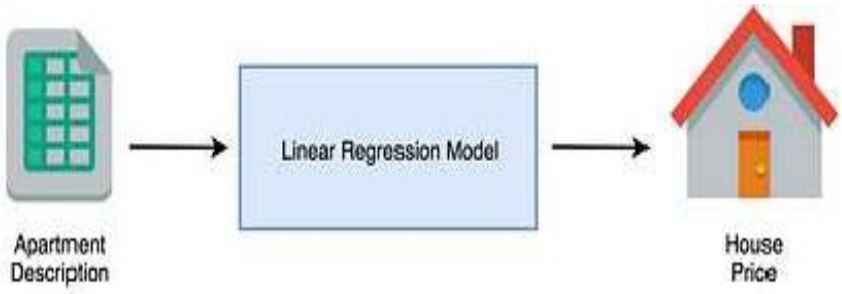
Linear Regression is a machine learning algorithm based on **supervised learning**. It performs a **regression task**. Regression models a target prediction value based on independent variables. It is mostly used for finding out the relationship between variables and forecasting. Different regression models differ based on – the kind of relationship between dependent and independent variables they are considering, and the number of independent variables getting used. There are many names for a regression's dependent variable. It may be called an outcome variable, criterion variable, endogenous variable, or regressed. The independent variables can be called exogenous variables, predictor variables, or regressor's.

Linear regression is used in many different fields, including finance, economics, and psychology,

to understand and predict the behavior of a particular variable. For example, in finance, linear regression might be used to understand the relationship between a company's stock price and its earnings, or to predict the future value of a currency based on its past performance.

It's a **Supervised Learning algorithm** which goal is to **predict continuous, numerical values based on given data input**. From the geometrical perspective, each data sample is a point. Linear Regression tries to **find parameters of the linear function**, so the **distance between the all the points and the line is as small as possible**. Algorithm used for parameters update is called **Gradient Descent**.

For example, if we have a dataset consisting of apartments properties and their prices in some specific area, Linear Regression algorithm can be used to find a mathematical function which will try to estimate the value of different apartment (outside of the dataset), based on its attributes.



Deep Neural Network :

A deep neural network (DNN) is an ANN with multiple hidden layers between the input and output layers. The main purpose of a neural network is to receive a set of inputs, perform progressively complex calculations on them, and give output to solve real wo ld problems like classification. We restrict ourselves to feed forward neural networks.

Neural networks are widely used in supervised learning and reinforcement learning problems. These networks are based on a set of layers connected to each other.

In deep learning, the number of hidden layers, mostly non-linear, can be large; say about 1000 layers. DL models produce much better results than normal ML networks.

A deep neural network is beneficial when you need to replace human labor with autonomous work without compromising its efficiency.

The deep neural network usage can find various applications in real life.

The American company Pony.ai is another example of how you can use DNN. They developed a system for AI cars that can work without a driver. It requires more than just a simple algorithm of actions, but a much deeper learning system, which should be able to recognize people, road signs and other markings like trees, and other important objects.

The famous company UbiTech creates AI robots. One of their creations is the Alpha 2 robot that can live in a family, speak with its members, search for information, write messages, and execute voice commands

The neural network needs to learn all the time to solve tasks in a more qualified manner or even to use various methods to provide a better result. When it gets new information in the system, it learns how to act accordingly to a new situation.

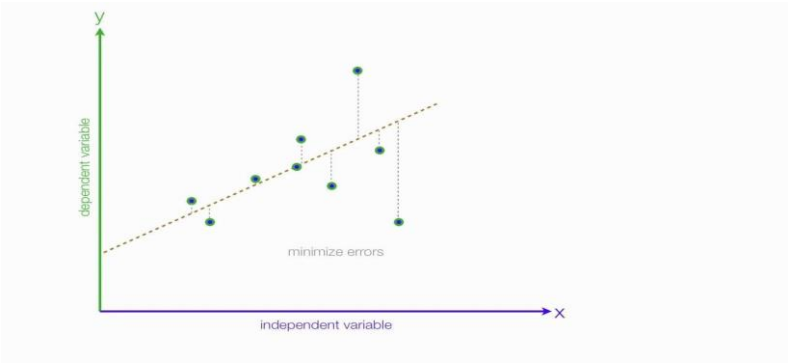
Learning becomes deeper when tasks you solve get harder. Deep neural network represents the type of machine learning when the system uses many layers of nodes to derive high-level functions from input information. It means transforming the data into a more creative and abstract component.

Visualising data

It is very important to always understand the structure of data. The more features there are, the harder it is. In this case, scatter plot is used to display the relationship between target and training features.

Depending on what is necessary to show, some other types of visualization (e.g. box plot) and techniques could be useful (e.g. clustering). Here, a linear dependency between features can be observed — with the increase of values on axis x, values on the y-axis are linearly increasing or decreasing accordingly. It's great because if that was not the case (e.g. relationship would be exponential), then it would be hard to fit a line through all the points and different algorithm should be considered.

The objective of Linear Regression is to find a line that minimizes the prediction error of all the data points.



- The Mean absolute error represents the average of the absolute difference between the actual and predicted values in the dataset. It measures the average of the residuals in the dataset.

$$MAE = \frac{1}{N}\sum_{i=1}^{N} |y_i - \hat{y}|$$

Where,

$\hat{y}$ — *predicted value of y*

$\bar{y}$ — *mean value of y*

- Mean Squared Error represents the average of the squared difference bet een the original and predicted values in the data set. It measures the variance of the residuals.

$$MSE = \frac{1}{N}\sum_{i=1}^{N} (y_i - \hat{y})^2$$

- Root Mean Squared Error is the square root of Mean Squared error. It measures the standard deviation of residuals.

$$RMSE = \sqrt{MSE} = \sqrt{\frac{1}{N}\sum_{i=1}^{N} (y_i - \hat{y})^2}$$

- The coefficient of determination or R-squared represents the proportion of the variance in the dependent variable which is explained by the linear regression model. It is a scale-free score i.e. irrespective of the values being small or large, the value of R square will be less than one.

$$R^2 = 1 - \frac{\sum(y_i - \hat{y})^2}{\sum(y_i - \bar{y})^2}$$

Boston House Price Prediction Dataset-

Boston House Price Prediction is a well-known dataset in machine learning and is often used to demonstrate regression analysis techniques. The dataset contains information about 506 houses in Boston, Massachusetts, USA. The goal is to predict the median value of owner-occupied homes in thousands of dollars.

The dataset includes 13 input features, which are:

CRIM: per capita crime rate by town

ZN: proportion of residential land zoned for lots over 25,000 sq.ft.

INDUS: proportion of non-retail business acres per town

CHAS: Charles River dummy variable (1 if tract bounds river; 0 otherwise)

NOX: nitric oxides concentration (parts per 10 million)

RM: average number of rooms per dwelling

AGE: proportion of owner-occupied units built prior to 1940

DIS: weighted distances to five Boston employment centers

RAD: index of accessibility to radial highways

TAX: full-value property-tax rate per $10,000

PTRATIO: pupil-teacher ratio by town

B: 1000(Bk - 0.63)^2 where Bk is the proportion of black people by town

LSTAT: % lower status of the population

The **output variable** is the median value of owner-occupied homes in thousands of dollars (MEDV).

To predict the median value of owner-occupied homes, a regression model is trained on the dataset. The model can be a simple linear regression model or a more complex model, such as a deep neural network. After the model is trained, it can be used to predict the median value of owner-occupied homes based on the input features. The model's accuracy can be evaluated using metrics such as mean squared error or

mean absolute error.

The prices of the house indicated by the variable MEDV is our target variable and the remaining are the feature variables based on which we will predict the value of a house.

How Deep Neural Network Work:

Boston House Price Prediction is a common example used to illustrate how a deep neural network can work for regression tasks. The goal of this task is to predict the price of a house in Boston based on various features such as the number of rooms, crime rate, and accessibility to public transportation.

Here's how a deep neural network can work for Boston House Price Prediction:

1. **Loading the Boston housing dataset:**

   A. As you can see, you have 404 training samples and 102 test samples, each with 13 numerical features, such as per capita crime rate, average number of rooms per dwelling, accessibility to highways, and so on.

   B. The targets are the median values of owner-occupied homes, in thousands of dollars.

   C. The prices are typically between $10,000 and $50,000. If that sounds cheap, remember that this was the mid-1970s, and these prices aren't adjusted for inflation.

2. **Preparing the data:**

   A. It would be problematic to feed into a neural network values that all take wildly different ranges.

   B. The network might be able to automatically adapt to such heterogeneous data, but it would definitely make learning more difficult.

   C. A widespread best practice to deal with such data is to do feature-wise normalization: for each feature in the input data (a column in the input data matrix), you subtract the mean of the feature and divide by the standard deviation, so that the feature is centered around 0 and has a unit standard deviation. This is easily done in Numpy.

   D. Note that the quantities used for normalizing the test data are computed using the training data. You should never use in your workflow any quantity computed on the test data, even for something as simple as data normalization.

3. **Building your network:**

A. Because so few samples are available, you'll use a very small network with two hidden layers, each with 64 units.

B. In general, the less training data you have, the worse overfitting will be, and using a small network is one way to mitigate overfitting.

C. The network ends with a single unit and no activation (it will be a linear layer). This is a typical setup for scalar regression (a regression where you're trying to predict a single continuous value).

D. Applying an activation function would constrain the range the output can take; for instance, if you applied a sigmoid activation function to the last layer, the network could only learn to predict values between 0 and 1. Here, because the last layer is purely linear, the network is free to learn to predict values in any range.

E. Note that you compile the network with the mse loss function—mean squared error,the square of the difference between the predictions and the targets. This is a widely used loss function for regression problems.

F. You're also monitoring a new metric during training: mean absolute error (MAE). It's the absolute value of the difference between the predictions and the targets. For instance, an MAE of 0.5 on this problem would mean your predictions are off by $500 on average.

4. **Model training:** The model is then trained using the training set. During training, the weights and biases of the nodes are adjusted based on the error between the predicted output and the actual output. This is done using an optimization algorithm such as stochastic gradient descent.
5. **Model evaluation:** Once the model is trained, it is evaluated using the testing set. The performance of the model is measured using metrics such as mean squared error or mean absolute error.
6. **Model prediction**: Finally, the trained model can be used to make predictions on new data, such as predicting the price of a new house in Boston based on its features.

By using a deep neural network for Boston House Price Prediction, we can obtain accurate predictions based on a large set of input features. This approach is scalable and can be used for other regression tasks as well.

Conclusion: In this way, we had learned and implemented Linear Regression using Deep Neural Network for Boston housing price prediction problem.

Group 1

Assignment No. 2

Title: Binary classification using Deep NeuralNetworks

Problem Statement:

Classify movie reviews into positive" reviews and "negative" reviews, just based on the text content of the reviews .Use IMDB dataset.

Objective:

To be able to apply the technique of Deep Neural network to Classify movie reviews into positive reviews and "negative reviews on IMDB Dataset.

Hardware Requirements:

Operating System recommended :- 64-bit Open source Linux or its derivative,

Multi-core CPU

GPU (Optional but recommended for faster training)

Sufficient RAM to accommodate the dataset and model

Software Requirements:

Programming Languages: Object Oriented Languages C++/JAVA/PYTHON/R

Tensorflow, Keras for Deep Learning

Jupyter Notebook or any Python IDE

Theory:

Classification is a type of supervised learning in machine learning that involves categorizing data into predefined classes or categories based on a set of features or characteristics. It is used to predict the class of new, unseen data based on the patterns learned from the labeled training data. In classification, a model is trained on a labeled dataset, where each data point has a known class label. The model learns to associate the input features with the corresponding class labels and can then be used to classify new, unseen data. For example, we can use classification to identify whether an email is spam or not based on its content and metadata, to predict whether a patient has a disease based on their medical records and symptoms, or to classify images into different categories based on their visual features. Classification algorithms can vary in complexity, ranging from simple models such as decision trees and k-nearest neighbors to more complex models such as support vector machines and neural networks. The choice of algorithm

depends on the nature of the data, the size of the dataset, and the desired level of accuracy and interpretability. Classification is a common task in deep neural networks, where the goal is to predict the class of an input based on its features.

How Deep Neural Network Work on Classification?

Deep neural networks are commonly used for classification tasks because they can automatically learn extract relevant features from raw input data and map them to the correct output class. The basic architecture of a deep neural network for classification consists of three main parts: an input layer, one or more hidden layers, and an output layer. The input layer receives the raw input data, which is usually preprocessed to a fixed size and format. The hidden layers are composed of neurons that apply linear transformations and nonlinear activations to the input features to extract relevant patterns and representations. Finally, the output layer produces the predicted class labels, usually as a probability distribution over the possible classes. During training, the deep neural network learns to adjust its weights and biases in each layer to minimize the difference between the predicted output and the true labels. This is typically done by optimizing a loss function that measures the discrepancy between the predicted and true labels, using techniques such as gradient descent or stochastic gradient descent. One of the key advantages of deep neural networks for classification is their ability to learn hierarchical representations of the input data. In a deep neural network with multiple hidden layers, each layer learns to capture more complex and abstract features than the previous layer, by building on the representations learned by the earlier layers. This hierarchical structure allows deep neural networks to learn highly discriminative features that can separate different classes of input data, even when the data is highly complex or noisy. Overall, the effectiveness of deep neural networks for classification depends on the choice of architecture, hyperparameters, and training procedure, as well as the quality and quantity of the training data. When trained properly, deep neural networks can achieve state-of-the-art performance on a wide range of classification tasks, from image recognition to natural language processing.

IMDB Dataset-The IMDB dataset is a large collection of movie reviews collected from the IMDB website, which is a popular source of user-generated movie ratings and reviews. The dataset consists of 50,000 movie reviews, split into 25,000 reviews for training and 25,000 reviews for testing. Each review is represented as a sequence of words, where each word is represented by an integer index based on its frequency in the dataset. The labels for each review

are binary, with 0 indicating a negative review and 1 indicating a positive review. The IMDB dataset is commonly used as a benchmark for sentiment analysis and text classification tasks, where the goal is to classify the movie reviews as either positive or negative based on their text content. The dataset is challenging because the reviews are often highly subjective and can contain complex language and nuances of meaning, making it difficult for traditional machine learning approaches to accurately classify them. Deep learning approaches, such as deep neural networks, have achieved state-of-the-art performance on the IMDB dataset by automatically learning to extract relevant features from the raw text data and map them to the correct output class. The IMDB dataset is widely used in research and education for natural language processing and machine learning, as it provides a rich source of labeled text data for training and testing deep learning models.

Observations: (To be written during or after implementing the problem statement)

1. Data preprocessing

2. Exploratory Data Analysis

3. Preparing the data for training the model

4. Splitting the data into training and testing sets

5. Training and testing the model

6. Model evaluation

Conclusion: In this way, we had learned and implemented Binary Classification using Deep Neural Network for IMDB dataset.

Group 1

Assignment No. 3

Title: Use MNIST Fashion Dataset and create a classifier to classify fashion clothing into categories.

Objective: To be able to use MNIST Fashion Dataset and create a classifier to classify fashion clothing into categories using Convolution Neural Network.

Problem Statement:

Implement Boston housing price prediction problem by linear regression using Deep Neural network. Use Boston House price prediction dataset.

Hardware Requirements:

Operating System recommended:64-bit Open source Linux or its derivative,

Multi-core CPU

GPU (Optional but recommended for faster training)

Sufficient RAM to accommodate the dataset and model

Software Requirements:

Programming Languages: Object Oriented Languages C++/JAVA/PYTHON/R

Tensorflow, Keras for Deep Learning

Jupyter Notebook or any Python IDE

Theory:

Classification is a type of supervised learning in machine learning that involves categorizing data into predefined classes or categories based on a set of features or characteristics. It is used to predict the class of new, unseen data based on the patterns learned from the labeled training data.

In classification, a model is trained on a labeled dataset, where each data point has a

known class label. The model learns to associate the input features with the corresponding class labels and can then be used to classify new, unseen data.

For example, we can use classification to identify whether an email is spam or not based on its content and metadata, to predict whether a patient has a disease based on their medical records and symptoms, or to classify images into different categories based on their visual features.

Classification algorithms can vary in complexity, ranging from simple models such as decision trees and k-nearest neighbors to more complex models such as support vector machines and neural networks. The choice of algorithm depends on the nature of the data, the size of the dataset, and the desired level of accuracy and interpretability.

Example- Classification is a common task in deep neural networks, where the goal is to predict the class of an input based on its features. Here's an example of how classification can be performed in a deep neural network using the popular MNIST dataset of handwritten digits.

The MNIST dataset contains 60,000 training images and 10,000 testing images of handwritten digits from 0 to 9. Each image is a grayscale 28x28 pixel image, and the task is to classify each image into one of the 10 classes corresponding to the 10 digits.

We can use a convolutional neural network (CNN) to classify the MNIST dataset. A CNN is a type of deep neural network that is commonly used for image classification tasks.

Convolutional Neural Networks (CNNs) are commonly used for image classification tasks, and they are designed to automatically learn and extract features from input images. Let's consider an example of using a CNN to classify images of handwritten digits.

In a typical CNN architecture for image classification, there are several layers, including convolutional layers, pooling layers, and fully connected layers. Here's a diagram of a simple CNN architecture for the digit classification task:

The input to the network is an image of size 28x28 pixels, and the output is a probability distribution over the 10 possible digits (0 to 9).

The convolutional layers in the CNN apply filters to the input image, looking for specific patterns and features. Each filter produces a feature map that highlights areas of the image that match the filter. The filters are learned during training, so the network can automatically learn which features are most relevant for the classification task.

The pooling layers in the CNN down sample the feature maps, reducing the spatial dimensions of the data. This helps to reduce the number of parameters in the network, while also making the features more robust to small variations in the input image.

The fully connected layers in the CNN take the flattened output from the last pooling layer and perform a classification task by outputting a probability distribution over the 10 possible digits.

During training, the network learns the optimal values of the filters and parameters by minimizing a loss function. This is typically done using stochastic gradient descent or a similar optimization algorithm.

Once trained, the network can be used to classify new images by passing them through the network and computing the output probability distribution.

Overall, CNNs are powerful tools for image recognition tasks and have been used successfully in many applications, including object detection, face recognition, and medical image analysis.

CNNs have a wide range of applications in various fields, some of which are:

Image classification: CNNs are commonly used for image classification tasks, such as identifying objects in images and recognizing faces.

Object detection: CNNs can be used for object detection in images and videos, which involves identifying the location of objects in an image and drawing bounding boxes around them.

Semantic segmentation: CNNs can be used for semantic segmentation, which involves partitioning an image into segments and assigning each segment a semantic label (e.g., "road", "sky", "building").

Natural language processing: CNNs can be used for natural language processing tasks, such as sentiment analysis and text classification.

Medical imaging: CNNs are used in medical imaging for tasks such as diagnosing diseases from X-rays and identifying tumors from MRI scans.

Autonomous vehicles: CNNs are used in autonomous vehicles for tasks such as object detection and lane detection.

Video analysis: CNNs can be used for tasks such as video classification, action recognition, and video captioning.

Overall, CNNs are a powerful tool for a wide range of applications, and they have been used successfully in many areas of research and industry.

How Deep Neural Network Work on Classification using CNN-

Deep neural networks using CNNs work on classification tasks by learning to automatically extract features from input images and using those features to make predictions. Here's how it works:

Input layer: The input layer of the network takes in the image data as input.

Convolutional layers: The convolutional layers apply filters to the input images to extract relevant features. Each filter produces a feature map that highlights areas of the image that match the filter.

Activation functions: An activation function is applied to the output of each convolutional layer to introduce non-linearity into the network.

Pooling layers: The pooling layers downsample the feature maps to reduce the spatial dimensions of the data.

Dropout layer: Dropout is used to prevent overfitting by randomly dropping out a percentage of the neurons in the network during training.

Fully connected layers: The fully connected layers take the flattened output from the last pooling layer and perform a classification task by outputting a probability distribution over the possible classes.

Softmax activation function: The softmax activation function is applied to the output of the last fully connected layer to produce a probability distribution over the possible classes.

Loss function: A loss function is used to compute the difference between the predicted probabilities and the actual labels.

Optimization: An optimization algorithm, such as stochastic gradient descent, is used to minimize the loss function by adjusting the values of the network parameters.

Training: The network is trained on a large dataset of labeled images, adjusting the values of the parameters to minimize the loss function.

Prediction: Once trained, the network can be used to classify new images by passing them through the network and computing the output probability distribution.

MNIST Dataset-

The MNIST Fashion dataset is a collection of 70,000 grayscale images of 28x28 pixels, representing 10 different categories of clothing and accessories. The categories include T-shirts/tops, trousers, pullovers, dresses, coats, sandals, shirts, sneakers, bags, and ankle boots.

The dataset is often used as a benchmark for testing image classification algorithms, and it is considered a more challenging version of the original MNIST dataset which contains handwritten digits. The

MNIST Fashion dataset was released by Zalando Research in 2017 and has since become a popular dataset in the machine learning community.

The MNIST Fashion dataset is a collection of 70,000 grayscale images of 28x28 pixels each. These images represent 10 different categories of clothing and accessories, with each category containing 7,000 images.

The images were obtained from Zalando's online store and are preprocessed to be normalized and centered. The training set contains 60,000 images, while the test set contains 10,000 images. The goal of the dataset is to accurately classify the images into their respective categories.

The MNIST Fashion dataset is often used as a benchmark for testing image classification algorithms, and it is considered a more challenging version of the original MNIST dataset which contains handwritten digits. The dataset is widely used in the machine learning community for research and educational purposes.

Conclusion: In this way, we had classified fashion clothing into various categories using CNN.

Group 2

Assignment No. 1

Title: Implement Human Face Recognition

Objectives: To build a deep neural network model that can recognize the identities of celebrities in the "CelebA" dataset.

Problem Statement:

Implement Boston housing price prediction problem by Linear regression using Deep Neural network. Use Boston House price prediction dataset.

Software and Hardware Requirements: Operating System recommended :- 64-bit Open source Linux or its derivative, Programming Languages: Object Oriented Languages C++/JAVA/PYTHON/R

Theory:

Human face recognition using deep neural networks involves building a neural network model that can take an image of a human face as input and accurately recognize the person in the image. Here is a sample code using TensorFlow to implement a basic face recognition system using a deep neural network: Human face recognition using deep neural networks (DNNs) involves training a neural network to identify and distinguish between different faces. The process typically involves the following steps: Data collection: A large dataset of face images is collected, including images of different individuals and under different lighting and pose conditions. Data preprocessing: The face images are preprocessed to remove noise, align the faces, and normalize the illumination. Feature extraction: The preprocessed face images are then fed into a deep neural network to extract high-level features that capture the important characteristics of a face. The neural network typically consists of several layers of convolutional and pooling operations, followed by fully connected layers that produce a feature vector. SNJB's Late Sau. K.B. Jain College Of Engineering Training: The extracted features are then used to train the neural network to distinguish between different faces. This is typically done using a supervised learning approach, where the network is trained on a labeled dataset of face images and their corresponding identities. Testing: After the neural network has been trained, it can be tested on a separate dataset to evaluate its performance. This typically involves measuring the accuracy of the network in correctly identifying the individuals in the test dataset.

Deployment: Once the neural network has been trained and tested, it can be deployed in a real-world application for face recognition. This typically involves capturing a face image, preprocessing it, and then feeding it into the neural network to obtain a feature vector. The feature vector is then compared to a database of known faces to determine the identity of the individual in the image. Overall, human face recognition using DNNs is a complex process that requires a large amount of data, sophisticated neural network architectures, and careful preprocessing and training. However, with the increasing availability of large datasets and powerful computing resources, DNN-based face recognition systems have become increasingly accurate and effective in real-world applications. Example- One example of human face recognition using DNNs is the FaceNet algorithm, which was developed by researchers at Google in 2015. FaceNet is a deep neural network that is trained to directly optimize the embedding of face images into a high-dimensional feature space, where distances between faces correspond to their similarity. The FaceNet architecture consists of a deep convolutional neural network that takes a raw face image as input and produces a 128-dimensional feature vector as output. The network is trained on a large dataset of face images with labels that correspond to different individuals. During training, the FaceNet network is optimized to minimize the distance between the feature vectors of images that depict the same person and maximize the distance between feature vectors of images that depict different people. This is done using a loss function called the triplet loss, which compares the distance between the feature vectors of an anchor image, a positive image (of the same person as the anchor), and a negative SNJB's Late Sau. K.B. Jain College Of Engineering image (of a different person). The goal is to minimize the distance between the anchor and positive images while maximizing the distance between the anchor and negative images. After training, the FaceNet network can be used for face recognition by capturing a face image, preprocessing it, and then feeding it into the network to obtain a 128-dimensional feature vector. This feature vector is then compared to a database of known faces by computing the Euclidean distance between the feature vector and the feature vectors of the faces in the database. The closest matching face in the database is then returned as the recognized identity. Overall, FaceNet is an example of a DNN-based face recognition system that has achieved high accuracy and robustness in real-world applications. It has been used in a variety of applications, including security systems, access control, and social media tagging.

Conclusion: In this way, we had implemented human face recognition system using deep neural networks.

Group 2

Assignment No. 2

Title: Implement Gender and Age Detection: predict if a person is a male or female and also their age

Objectives: To build a deep neural network model that can recognize Gender and Age Detection on UTKFace dataset.

Theory:

Collect and prepare the dataset: In this case, we can use the "UTKFace" dataset which contains images of faces with their corresponding gender and age labels. We need to preprocess the data, like resizing the images to a uniform size, shuffling the dataset, and limit the age to a certain value (like 100 years). Split the dataset: Split the dataset into training, validation, and testing sets. The usual split ratio is 80%, 10%, and 10%, respectively. Define data generators: Define data generators for training, validation, and testing sets using the "ImageDataGenerator" class in Keras. This class provides data augmentation techniques that can improve the model's performance, such as rotation, zoom, and horizontal flip. Define the neural network model: Define a convolutional neural network (CNN) model that takes the face images as input and outputs two values - the probability of being male and the predicted age. The model can have multiple convolutional and pooling layers followed by some dense layers. SNJB's Late Sau. K.B. Jain College Of Engineering Compile the model: Compile the model with appropriate loss and metrics for each output (gender and age). In this case, we can use binary cross-entropy loss for gender and mean squared error (MSE) for age. Train the model: Train the model using the fit method of the model object. We need to pass the data generators for the training and validation sets, as well as the number of epochs and batch size. Evaluate the model: Evaluate the model's performance on the testing set using the evaluate method of the model object. This will give us the accuracy and mean absolute error (MAE) of the model. Predict the gender and age of a sample image: Load a sample image and preprocess it. We can use the "cv2" library to read the image, resize it to the same size as the training images, and normalize it. Then, we can use the "predict" method of the model object to get the predicted gender and age.

Conclusion: In this way, we had implemented gender and age detection system using deep neural networks.

INSTITUTE VISION-MISSION

Our Vision

- The institute envisions to develop itself into a center of academic excellence in the field of Engineering and Management education in order to develop future technocrats and managers having right knowledge, skill and attitude to serve the society and industries to fulfill their ever changing requirements.

Our Mission

- To train our students to become best Engineering Entrepreneurs today, who will lead the organizations successfully into the future; locally, nationally and globally.
- To provide an environment which fosters continuous improvement & innovation with related technical support & facilities to enhance student and faculty effectiveness.
- To provide programs focusing on the holistic development of the individual with the emphasis on personality grooming, physical fitness and a strong sense of social and environmental responsibility.
- To improve logic & scientific reasoning and to develop global mindset amongst the students and prepare them to work in heterogeneous environment.

COMPUTER DEPARTMENT VISION-MISSION

Our Vision

- To become an acclaimed center of excellence by rendering modern technology, academics and research for creating holistic, socio-economic professionals with interdisciplinary potential.

Our Mission

- To foster [illegible] creative [illegible] cademia [illegible] ftware

awareness of ethical and social responsibilities.